

A Frozen Socket for Neural Specialists

Unrelated Skills Compile into a Typed Packet ABI

github.com/Dinkum (Blake)

2026-05-06

Abstract

Modern AI systems are starting to look less like single monoliths and more like collections of specialists: a retriever here, a verifier there, a ranker, a planner, a graph solver, a calculator. The awkward part is that the usual interfaces are still human-shaped. We make neural components talk through text, JSON, labels, logits, or raw hidden states, even when the receiver is another neural component.

This paper tests a small alternative: a frozen neural socket. The socket is trained once from two source specialist families, modular checksum and dot-product retrieval. Then the socket stops learning. New neural skills can only train local encoders into the frozen packet ABI. The packet is tiny: two code slots, each choosing one of 32 symbols. The socket’s frozen readers interpret packets as a typed state and a confidence margin.

The surprising part is not that held-out retrieval still works. The surprising part is that unrelated skills work. A graph shortest-path specialist plugs into the frozen socket at 99.7% state and 99.6% margin transfer. A max-index specialist reaches 95.0% state transfer. Counting positive entries reaches 93.3%. Ranking the first item reaches 92.5%. These skills were not source domains for the socket.

A matched random socket does not explain the result. Across unrelated held-out skills, the trained socket beats the random socket by 43.4 percentage points on the composite transfer score. The random control sees the same specialists, the same local adapter protocol, and the same packet size. It lacks only source-trained socket semantics.

The claim is deliberately scoped: a small frozen typed ABI can act as a reusable socket for neural specialists. New skills do not need to retrain the socket. They compile into it.

1. Introduction

Software got powerful when components stopped needing to know each other's insides. A program can call a file system, a graphics driver, or a network stack because there is an interface in the middle. The interface is not the whole computation. It is the contract.

Neural systems mostly do not have that object yet.

When we connect models today, we usually pick one of four awkward options. Text is flexible but bulky and lossy. JSON is cleaner, but still a human-facing serialization. Logits and labels are narrow. Hidden states are rich, but private to one model's coordinate system. None of these quite feels like a native socket for neural skills.

This paper asks a blunt question:

Can a small frozen neural interface behave like a reusable socket for unrelated skills?

The setup is intentionally concrete. We train a packet ABI on two source specialist families:

- modular checksum specialists
- dot-product retrieval specialists

The ABI learns to encode each specialist's hidden features into two discrete packet slots. Frozen readers attach typed meaning to the packet:

- a state/class
- a confidence or margin bucket

Then we freeze the ABI. No more socket learning.

Now come the new skills. They are allowed to train only local encoders into the frozen socket. The socket readers do not move. The source-trained semantic heads do not move. If a new skill works, it is because the specialist learned to compile its private computation into an existing packet contract.

The held-out skills are deliberately not all cosmetic variants of the source tasks:

skill	input shape	state meaning	why it matters
held-out retrieval	query and candidates	winning candidate bucket	source-family reuse
different retrieval architecture	query and candidates	winning candidate bucket	not one computation graph
max-index	real-valued list	largest slot	new task family
count-positive	real-valued list	number above zero	counting
rank-first	real-valued list	rank of first item	ordering

skill	input shape	state meaning	why it matters
graph shortest-path	graph adjacency matrix	path length bucket	structural graph reasoning

The graph row is the cleanest layman hook. A tiny socket trained on checksum and retrieval can later accept a specialist that reasons over paths in a graph.

That should not be assumed. A frozen random socket is also a fixed interface. A strong enough local encoder might bend itself into random labels. So we run a matched random-socket control: same packet size, same specialists, same adapters, same evaluation, but no source-trained ABI semantics.

It fails. The trained socket wins broadly.

A frozen neural socket for new skills

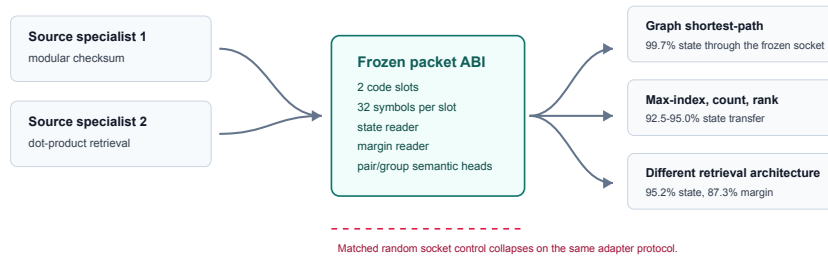


Figure 1: A frozen packet ABI trained on source specialists becomes the socket. Later specialists train only local encoders into that frozen socket.

1.1 Contributions

- We test a frozen typed packet ABI into which later specialists train only local encoders.
- We evaluate transfer to skills that are visibly different from the source domains, including graph shortest-path, max-index, counting, and ranking.
- We compare against a matched random-socket control, not only shuffled-packet controls.
- We show broad trained-over-random lift across unrelated skills: 43.4 percentage points on the mean composite score.
- We argue for a narrow but vivid claim: small neural specialists can compile into a frozen typed socket without sharing architecture, task family, or weights.

2. Related Work

This experiment sits near several older ideas, but it is not quite any of them.

Neural module networks compose learned modules into larger systems, often using a question parse or task structure to decide which modules to instantiate [1]. That line of work is about composing networks. The present experiment is about a reusable interface between independently trained specialists.

Emergent communication studies show that neural agents can learn messages for cooperation [2,3]. Those settings often optimize a sender and receiver together for a task. Here the key constraint is different: after source training, the receiver-side socket is frozen. Later senders must compile into an already existing ABI.

Discrete latent models such as VQ-VAE show that neural systems can use learned codebooks as compact bottlenecks [4]. Our packet is also discrete and low-bandwidth, but it is not just a reconstruction latent. It is a typed interface with frozen semantic readers.

Representation similarity and model stitching ask whether internal representations can be compared, aligned, or interchanged across networks [5,6,7]. This paper is close in spirit, but the target is operational rather than diagnostic: can a later specialist plug into a frozen semantic socket and be read correctly?

Task arithmetic shows that model behaviors can sometimes be edited by moving in weight space [8]. That is another path to modularity. Here the modular object is not a weight delta. It is a tiny runtime packet interface.

The useful comparison is a software ABI. A normal component does not need to know the internal implementation of the operating system. It needs to compile calls to the right interface. This paper asks whether a small neural analogue can exist in a controlled setting.

3. The Frozen Socket Test

The packet ABI has two slots. Each slot chooses one of 32 symbols. The whole message is therefore a tiny discrete object, but it is not treated as a raw class label. The ABI has frozen readers:

- a state reader
- a margin reader
- downstream semantic heads trained on source packets for pair and group relations

The state is the task’s discrete answer bucket. The margin is a confidence-like ambiguity bucket. Different tasks can instantiate these types differently. In retrieval, state is the winning candidate bucket and margin is the top1/top2 gap. In graph shortest-path, state is the path-length bucket and margin is graph density.

This typed contract is the entire trick. The socket is not asked to carry arbitrary thought. It is asked to carry “what state is this, and how confident or separated is it?”

Training has three phases:

1. Train source and held-out specialists on their own tasks.
2. Train the packet ABI on source specialists only, then freeze it.
3. Train local encoders for held-out skills into the frozen ABI.

Evaluation then reads held-out packets using the frozen readers and source-trained semantic heads. We report:

- **state transfer:** frozen state reader accuracy on held-out skill packets
- **margin transfer:** frozen margin reader accuracy
- **pair transfer:** a source-trained four-way relation over two packets
- **group transfer:** a source-trained group/ranking relation over four packets

For each skill, we also evaluate shuffled packets. Shuffling preserves the broad evaluation distribution while breaking symbol identity. More importantly, we run a random-socket control: the ABI is randomly initialized and frozen, then the same adapter protocol is used.

Nominal chance levels are 12.5% for state, 16.7% for margin, 25% for pair, and 25% for group. The random-socket control is the stronger baseline because it captures what a flexible local adapter can do without a learned socket contract.

4. Results

The headline result is broad and easy to read. The trained socket is not merely better on the source-like rows. It beats the random socket on every held-out skill, including the visually different ones.

The aggregate result over six retained runs is:

headline	value
mean held-out lift over random socket	43.4%
mean held-out state transfer	95.1%
mean held-out pair transfer	76.1%

The per-app ranking is:

skill	trained	random	lift	state
retrieval-arch	85.9%	39.6%	46.3%	95.2%
graph-path	86.9%	40.7%	46.1%	99.7%

skill	trained	random	lift	state
max-index	81.5%	36.0%	45.5%	95.0%
retrieval	79.8%	37.9%	41.8%	89.7%
count-positive	78.8%	38.8%	40.0%	93.3%
rank-first	78.4%	39.2%	39.1%	92.5%
checksum	51.6%	30.1%	21.5%	41.9%

The detailed report also tracks margin, pair, and group transfer. The main body keeps the table narrower because the pattern is already visible in the chart: trained sockets sit near 78-87% composite on the unrelated skills, while random sockets sit near 36-41%.

The checksum row is weakest. That is useful, because it keeps the story honest. This is not “everything becomes perfect.” The surprising result is that the unrelated rows are stronger than the obvious held-out checksum row.

Graph shortest-path is the best example. The specialist sees a graph adjacency matrix and predicts a path-length bucket. The frozen socket was trained on modular checksum and dot-product retrieval. Still, after local compilation, the frozen state reader reaches 99.7% and the frozen margin reader reaches 99.6%. The random socket’s composite score is 40.7%, while the trained socket reaches 86.9%.

Max-index gives a second simple picture. The input is a real-valued list, and the state is the largest slot. The trained socket reaches 95.0% state transfer and 81.5% composite transfer. The random socket’s composite is 36.0%.

Counting and ranking also work. Count-positive reaches 93.3% state transfer. Rank-first reaches 92.5%. These are small algorithmic skills, but they are different enough to make the socket story vivid: the interface is carrying a typed answer, not memorizing one input format.

5. Why the Random Control Matters

The random control is the difference between a paper and a demo.

Without it, one could argue that local encoders are just powerful. Maybe any frozen decoder can be targeted if the adapter trains long enough. The random-socket control keeps that possibility alive and tests it directly.

The answer is clear. Random sockets can sometimes pick up weak structure, but they do not behave like reusable ABIs. They sit around 36-41% composite on the unrelated skills, while the trained socket sits around 78-87%.

The most important comparisons are:

Unrelated skills compile into the trained socket

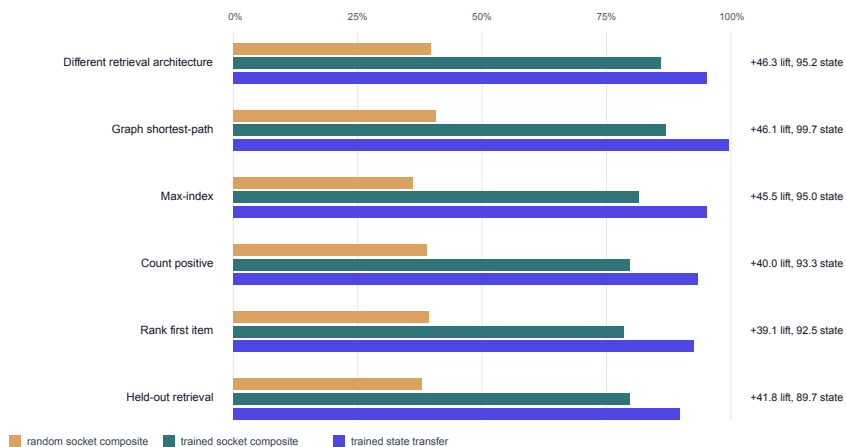


Figure 2: Composite transfer and state transfer for held-out skills. The trained socket wins broadly over a matched random frozen socket.

skill	trained	random	state gap
graph-path	86.9%	40.7%	99.7% vs 48.1%
max-index	81.5%	36.0%	95.0% vs 55.4%
count-positive	78.8%	38.8%	93.3% vs 45.4%
rank-first	78.4%	39.2%	92.5% vs 51.8%

This is the central empirical claim. The source-trained socket has learned a reusable semantic target. Later specialists can compile into that target. Random frozen targets do not give the same behavior.

6. What the Socket Is Actually Learning

The exciting reading is that this looks like the beginning of a neural interface language. The technical claim we can defend is more specific: the socket learns a typed packet layout that later specialists can target.

- slot information sufficient for a state reader
- slot information sufficient for a margin reader
- semantic summaries that source-trained pair and group heads can reuse

The packet is closer to an ABI than to natural language: compact, typed, and stable enough for downstream readers to keep working.

That distinction matters. A software ABI is boring in the best possible way. It says: put this kind of thing here, put that kind of thing there, and downstream code can keep working. This experiment suggests that a neural analogue can be learned in miniature. The symbols are not manually assigned, but after training they behave like a stable contract.

The pair and group results are important because they test more than direct decoding. A direct state reader could be a narrow classifier. Pair transfer asks whether source-trained relations over packets still work on held-out skills. Group transfer asks whether a source-trained relation over multiple packets survives.

Pair transfer is strong: 74.0% on graph shortest-path, 81.3% on max-index, 71.6% on count-positive, and 73.5% on rank-first. Group transfer is also positive, but noisier. I would not call this a full packet algebra yet. I would call it a typed socket with early semantic composability.

7. Limitations

The domains are controlled. That is by design. Controlled domains make it possible to know exactly what state and margin mean, and to run clean random-socket and shuffled-packet controls. The result should not be read as a claim about open-ended natural-language agents.

The socket is supervised. It is trained with state and margin targets. The paper does not claim unsupervised discovery of meaning.

The specialists are small. Larger models may make the adapter problem easier, harder, or stranger.

The packet has a narrow type system. It carries state and confidence-like margin well. Pair semantics transfer. Group semantics are positive but less crisp. A larger neural socket system would need richer types, versioning, error handling, and probably many sockets, not one two-slot packet.

Finally, this paper tests compatibility after local compilation. It does not show that unrelated models naturally share packet coordinates before adaptation. The point is precisely that they do not need to. The specialist compiles into the socket.

8. Conclusion

A collection of neural specialists needs more than specialists. It needs a socket.

This paper shows a small one. Train a frozen packet ABI on checksum and retrieval. Freeze it. Then plug in a graph shortest-path specialist, a max-index specialist, a counting specialist, a ranking specialist, and a different retrieval architecture. Train only local encoders. The frozen socket still reads typed

state and confidence, and source-trained semantic heads still work above chance by large margins.

The result is not a universal neural language. It is more concrete: a reusable neural ABI with a tiny packet, a typed contract, and evidence that unrelated specialists can compile into it.

That is the interesting object. Not one model doing everything. Not models chatting in English. A little socket in the middle, stable enough that new neural skills can plug in.

References

- [1] Jacob Andreas, Marcus Rohrbach, Trevor Darrell, and Dan Klein. “Neural Module Networks.” arXiv:1511.02799, 2015.
- [2] Jakob N. Foerster, Yannis M. Assael, Nando de Freitas, and Shimon Whiteson. “Learning to Communicate with Deep Multi-Agent Reinforcement Learning.” arXiv:1605.06676, 2016.
- [3] Mycal Tucker, Huao Li, Siddharth Agrawal, Dana Hughes, Katia Sycara, Michael Lewis, and Julie Shah. “Emergent Discrete Communication in Semantic Spaces.” arXiv:2108.01828, 2021.
- [4] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. “Neural Discrete Representation Learning.” arXiv:1711.00937, 2017.
- [5] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. “Similarity of Neural Network Representations Revisited.” arXiv:1905.00414, 2019.
- [6] Yamini Bansal, Preetum Nakkiran, and Boaz Barak. “Revisiting Model Stitching to Compare Neural Representations.” arXiv:2106.07682, 2021.
- [7] Adriano Hernandez, Rumen Dangovski, Peter Y. Lu, and Marin Soljacic. “Model Stitching: Looking For Functional Similarity Between Representations.” arXiv:2303.11277, 2023.
- [8] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. “Editing Models with Task Arithmetic.” arXiv:2212.04089, 2022.